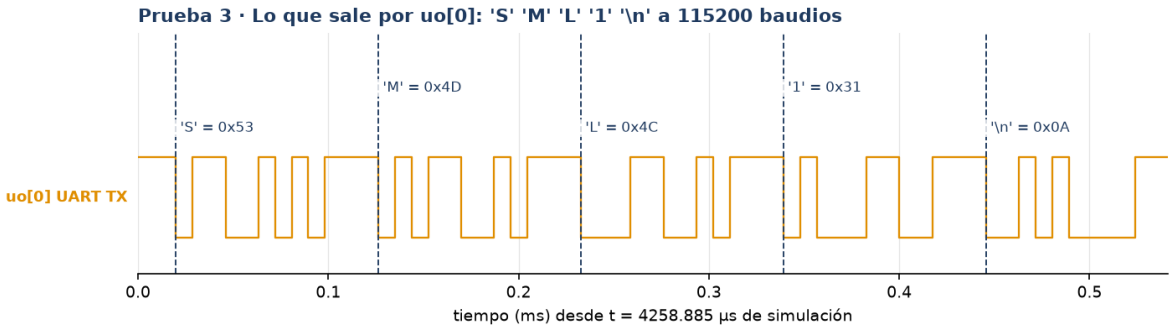
Informe final del chip SERV TinyML

Qué estaba mal, cómo lo arreglé y todas las pruebas que lo demuestran



Diseño físico 3x2 en GitHub Actions: APROBADO (gds, precheck y gl_test en verde)

Repositorio github.com/ric1806/tiny, rama claude/hola-wl31sp · commit final c853560 · 28 de septiembre de 2026

Contenido

1. Resumen 3
2. Qué estaba mal y cómo lo arreglé 4
3. Cómo probé el chip (el montaje) 6
4. Las regresiones originales 7
5. Las pruebas del chip completo 7
6. Gate-level 11
7. Área, tiempos y lint 12
8. Lo que no se pudo probar aquí 12
9. Entregables 12
10. Cómo repetir las pruebas 13

1. Resumen

Mantuve el tamaño **3x2**, como pediste. El chip cabía, pero el diseño físico fallaba por tiempos. Encontré la causa, la corregí y verifiqué el resultado con las pruebas originales, con pruebas nuevas y con el diseño físico completo en GitHub Actions.

Prueba	Qué verifica	Resultado
Diseño físico 3x2 (LibreLane en GitHub Actions)	Colocación, reloj, ruteo, tiempos en todas las esquinas, DRC, LVS, antenas	**Aprobado** (1 aviso: setup -9,5 ps en la esquina más extrema)
Precheck de Tiny Tapeout	Reglas de entrega de Tiny Tapeout	**Aprobado**
Gate-level oficial (`gl_test`)	Prueba del top module sobre el netlist final, después del ruteo	**Aprobado**
10 regresiones originales	MAC, SPI, puente de memoria, SERV, arranque desde Flash/PSRAM	**10/10 PASS**
5 pruebas de placa (RTL)	El chip completo visto solo por sus pines, con la configuración real	**5/5 PASS**
Las mismas 5 en gate-level (síntesis)	Sobre el netlist de 8 108 compuertas Sky130	**5/5 PASS**
Lint (Verilator)	Errores de código	**0 errores**, 5 avisos inofensivos
Chip real / FPGA	Funcionamiento eléctrico	**Requiere hardware**

Lo que cambié

1. **Quité la compuerta que pausaba el reloj de SERV.** Creaba un segundo reloj desfasado que hacía fallar el hold. SERV ya sabía esperar a la memoria; el arranque funciona igual y tarda lo mismo.
2. **El MAC acumula en un ciclo aparte** (9 ciclos por multiplicación en vez de 8). Su camino más lento baja de 9,0 a 7,5 ns.
3. **Reparación de hold ajustada**: margen pequeño (0,02 ns) tras el árbol de reloj y verificación activada después del ruteo global, una etapa que venía apagada.
4. **Documentación alineada con 3x2** (README, PROJECT_GUIDE, TASKS, SKY130_MIGRATION, STUDENT_HANDOFF) y paso nuevo en el CI que imprime el resumen de tiempos.
5. **Pruebas:** la prueba de pausa de reloj pasó a comprobar la espera de SERV, y añadí 5 pruebas del chip completo que también corren en gate-level.

2. Qué estaba mal y cómo lo arreglé

2.1 El fallo en GitHub Actions

Con el diseño original en 3x2, el flujo LibreLane llegaba hasta el final: colocación, árbol de reloj, ruteo, DRC, LVS y antenas pasaban. Pero terminaba con este error:

```
ERROR  Hold violations found in the following corners:
       * max_ff_n40C_1v95
       * min_ff_n40C_1v95
       * nom_ff_n40C_1v95
WARNING  Setup violations found in the following corners:
       * max_ss_100C_1v60  * min_ss_100C_1v60  * nom_ss_100C_1v60
```

Traducido: en la **esquina rápida** de fabricación (*ff*, –40 °C, 1,95 V) algunas señales llegaban **demasiado pronto** (hold, error fatal), y en la **esquina lenta** (*ss*, 100 °C, 1,60 V) algunas llegaban **demasiado tarde** (setup, advertencia). El flujo había metido **725 celdas de retardo** para arreglar el hold, sin conseguirlo.

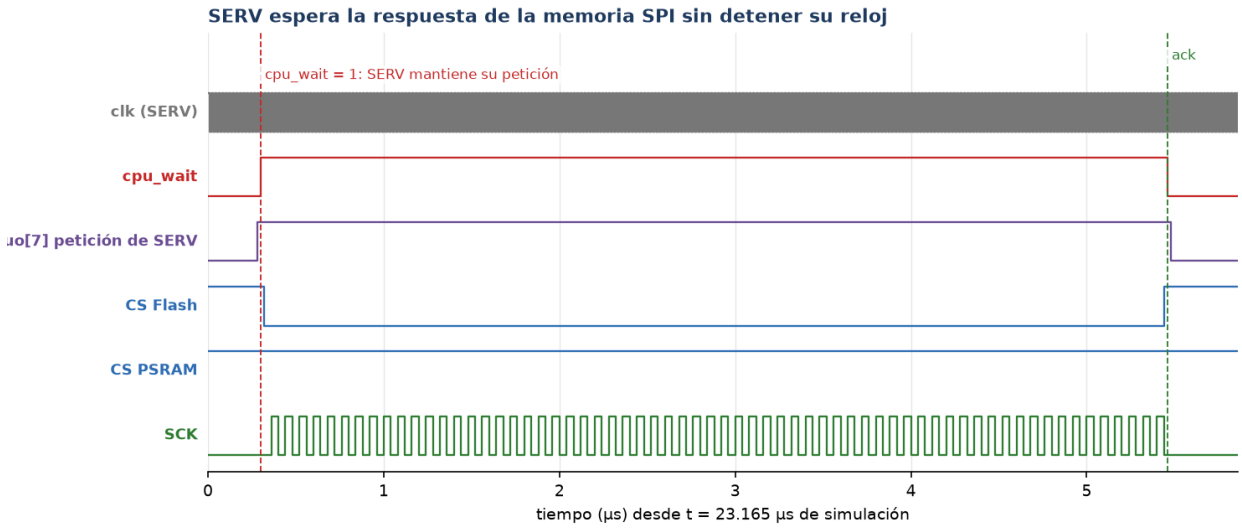
2.2 La causa: dos relojes desfasados

El log también decía que la red `cpu_clk` alimentaba a **1 191 flip-flops**. Esa red era el reloj de SERV **después** de la compuerta que lo pausaba. El resto del chip (puente SPI, MAC, UART) usaba `clk` directamente. Al pasar por la compuerta y por su propio árbol, `cpu_clk` llegaba con retraso, y en cada señal que cruzaba entre los dos relojes ese desfase se comía el margen de tiempo.

> **Analogía para entenderlo**
>
> Es como dos relojes de pared que deberían marcar la misma hora, pero uno va un poco atrasado. Si uno le pasa algo al otro "a las 3 en punto", el segundo todavía no ha llegado a las 3 y lo recibe antes de tiempo.

2.3 Arreglo 1: quitar la compuerta de reloj

La compuerta existía porque se creía que SERV no podía esperar a la memoria lenta. Lo comprobé en simulación: con la compuerta **desactivada**, el arranque completo desde Flash/PSRAM pasa y tarda **exactamente lo mismo** (2 655 240 ns). SERV espera solo, porque no avanza hasta recibir la respuesta (*ack*) de la memoria. Así que la quité: ahora todo el chip usa **un solo reloj**.



Ahora: mientras la memoria responde (cpu_wait = 1), SERV mantiene su petición (uo[7]) y su reloj sigue funcionando.

2.4 Arreglo 2: acortar el camino más lento del MAC

El MAC hacía en el mismo ciclo la última suma de la multiplicación (16 bits), el cambio de signo y la suma de 32 bits del acumulador. Ahora guarda el producto en un registro y lo acumula en el ciclo siguiente. El camino más lento estimado pasó de **9,04 ns a 7,49 ns**. Cuesta un ciclo más por multiplicación (9 en vez de 8), y el firmware no cambia porque ya esperaba a que el MAC terminara.

2.5 Arreglo 3: la reparación de hold

Con un solo reloj, el análisis tras construir el árbol de reloj ya decía **"No setup violations found"** y el peor hold era de solo –**0,047 ns**. Faltaba ajustar la reparación de hold, en tres pasos:

1. Con margen 0,1 ns (el valor por defecto) el flujo metió **1 590 celdas** (+17,9 % de área) y ya no cupieron (`DPL-0036`). Unos 1 100 flip-flops del banco de registros quedan casi en el límite, así que cualquier margen mayor de 0,02 ns cuesta más de mil celdas.
2. Con margen 0, todo pasaba menos **5 caminos** de SERV que fallaban hold por **11 picosegundos** en la esquina rápida, por un pequeño desfase (0,08 ns) entre ramas del árbol de reloj que la estimación temprana no ve.
3. Subir el margen "después del ruteo" no cambió nada. Leyendo el código de LibreLane descubrí por qué: esa etapa (`RUN_POST_GRT_RESIZER_TIMING`) **viene desactivada por defecto**. La configuración final pone 0,02 ns tras el árbol de reloj, que mete **76 celdas** (+0,9 %) y es lo que cerró el hold, y activa esa etapa con 0,01 ns como verificación con los cables reales: se ejecutó y no encontró nada que reparar.

2.6 Resultado final en GitHub Actions

Con los tres arreglos, el commit **c853560** pasó todo el flujo (run 36503628717): **gds**, **precheck** de Tiny Tapeout y **gl_test** (gate-level oficial) en verde; DRC, LVS y antenas limpios. La reparación de hold tras el árbol de reloj metió solo 76 celdas de retardo (+0,9 % de área), y la etapa tras el ruteo global (que venía desactivada y activé) no encontró nada más que reparar. Esta es la tabla de tiempos final en las 9 esquinas (holgura positiva = bien):

Esquina (proceso, temperatura, voltaje, cables)	Peor hold (ns)	Peor setup (ns)
nom_tt_025C_1v80 (típica)	0,1835	6,6756
nom_ss_100C_1v60 (lenta)	0,6057	0,5767
nom_ff_n40C_1v95 (rápida)	0,0102	8,4033
min_tt_025C_1v80	0,1817	6,8431
min_ss_100C_1v60	0,6024	0,9502
min_ff_n40C_1v95	0,0148	8,5143
max_tt_025C_1v80	0,1803	6,5331
max_ss_100C_1v60 (la más extrema)	0,6092	**-0,0095**
max_ff_n40C_1v95	0,0046	8,3104

Hold: cerrado en todas las esquinas. **Setup:** positivo en 8 de 9; en la esquina más extrema (lenta, 100 °C, 1,60 V y cables en el peor caso) un único camino llega 9,5 ps tarde a 50 MHz. Equivale a funcionar a 49,98 MHz en vez de 50 en esas condiciones límite; LibreLane lo reporta como aviso, no como error. También quedan avisos de *max slew* y *max capacitance* (transiciones lentas en algunos cables), sobre todo en la esquina lenta; tampoco detienen el flujo. **Nota:** el "viewer" (visor 3D en GitHub Pages) falla porque Pages no está activado en el repositorio; no afecta al chip.

3. Cómo probé el chip (el montaje)

Un **testbench** es un circuito "de mentira" que rodea a tu diseño para probarlo: le da reloj, lo resetea, le conecta memorias simuladas y revisa qué sale por sus pines.

Pieza	Qué es
Diseño bajo prueba	El top module completo con sus parámetros de producción: reloj 50 MHz, UART 115200 baudios, SCK 12,5 MHz.
Memoria simulada	Modelo de Flash + PSRAM (comandos SPI 0x03 y 0x02) con el firmware `serv_alive.hex` cargado.
Entradas sin usar	Reciben valores **aleatorios** durante el arranque para demostrar que no afectan.
Simulador	Icarus Verilog 12 + cocotb 2.0.1.
Gate-level	Netlist de Yosys con la biblioteca `sky130_fd_sc_hd` y los modelos Verilog oficiales de cada compuerta.
Diseño físico	LibreLane (el flujo oficial de Tiny Tapeout) en GitHub Actions.

4. Las regresiones originales

Todas las pruebas que traía el proyecto pasan con el diseño final:

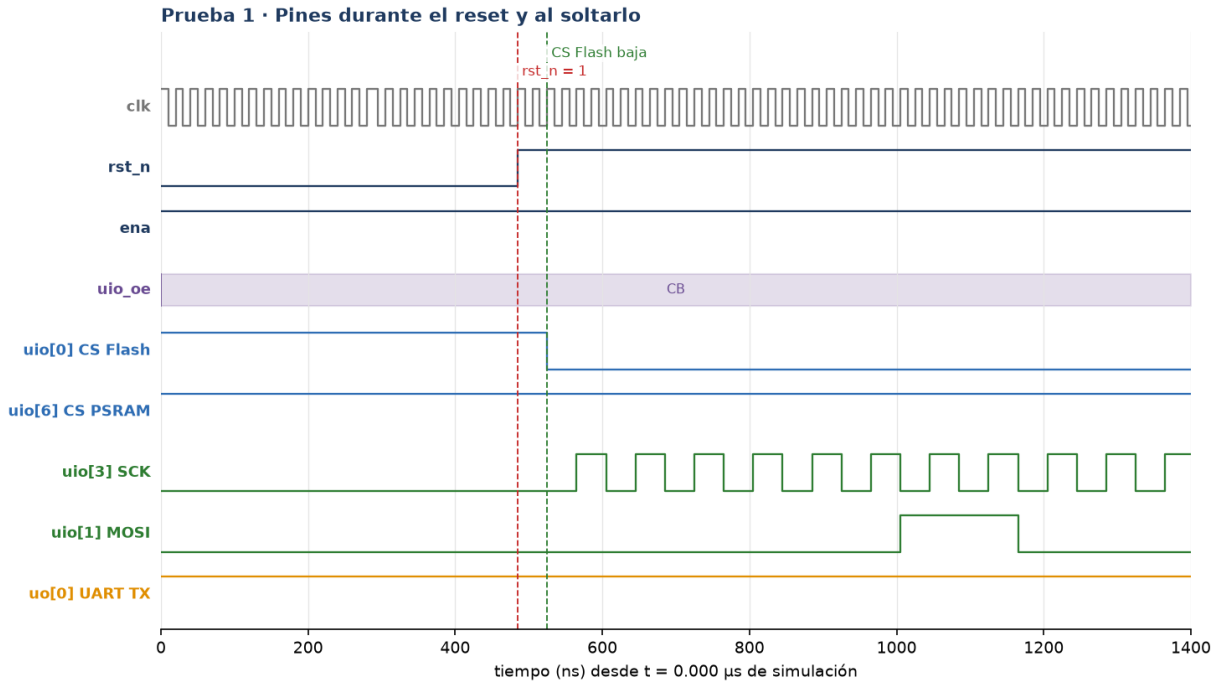
Grupo	Qué comprueba	Resultado
model/test_int8_reference.py	Referencia Python del MAC	**OK**
Makefile	Top module: pines de memoria en reset y con ena = 0	**1/1**
Makefile.mac	MAC por MMIO + 10 000 productos aleatorios contra Python	**2/2**
Makefile.spi	Lectura SPI de una palabra de la Flash	**1/1**
Makefile.mem	Puente: Flash o PSRAM, prioridad, escritura	**3/3**
Makefile.serv	SERV ejecuta el firmware C desde memoria ideal	**1/1**
Makefile.extmem	Arranque desde Flash/PSRAM + espera de SERV	**2/2**

> **Ojo**
>
> Una prueba cambió: la segunda de `test_serv_extmem.py` comprobaba que el reloj de SERV se detenía. Como la compuerta ya no existe, ahora comprueba lo que la sustituye: que SERV mantiene su petición estable, que no llega un `ack` antes de tiempo y que llega exactamente uno al final. La versión original está en el paquete de testbenches.

5. Las pruebas del chip completo

Cada prueba mira el chip **solo por sus pines**, como un analizador lógico en el laboratorio. Las imágenes son las formas de onda reales de la simulación del diseño final.

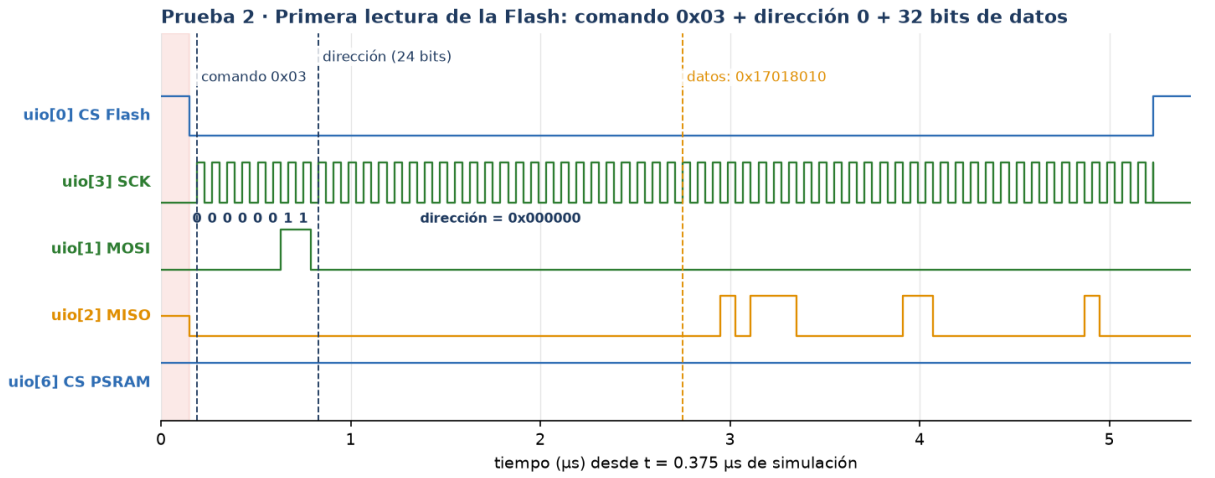
5.1 Pines seguros durante el reset



uio_oe = CB = 1100 1011. Los CS de memoria en 1 y SCK quieto hasta soltar el reset.

Resultado: **PASS**.

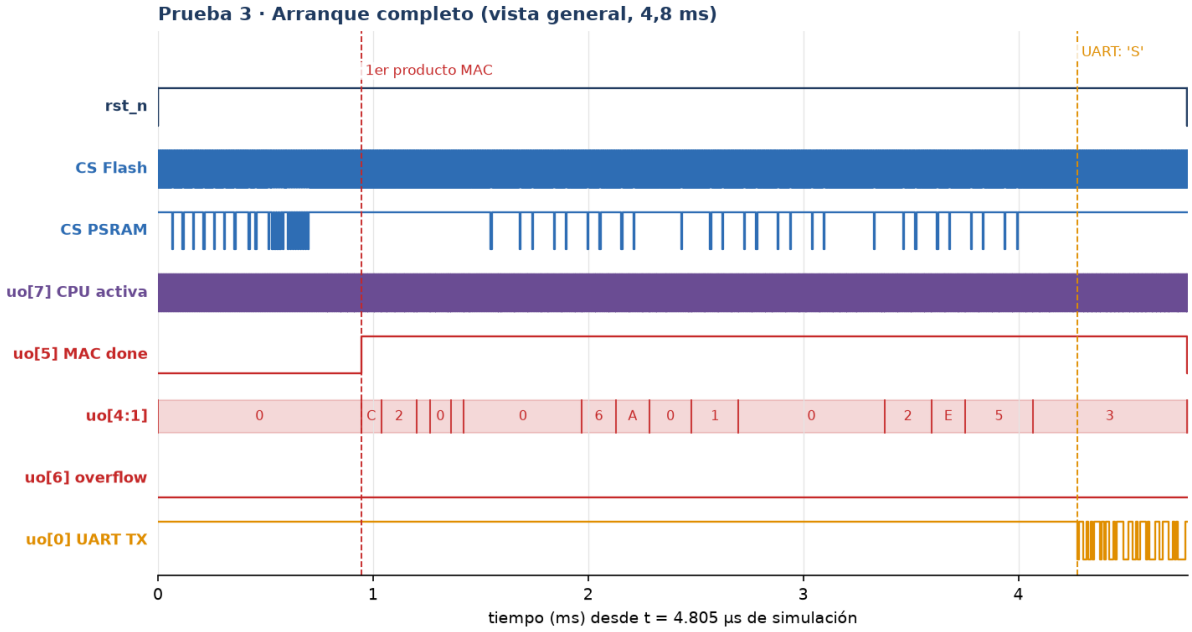
5.2 Primera lectura de la Flash



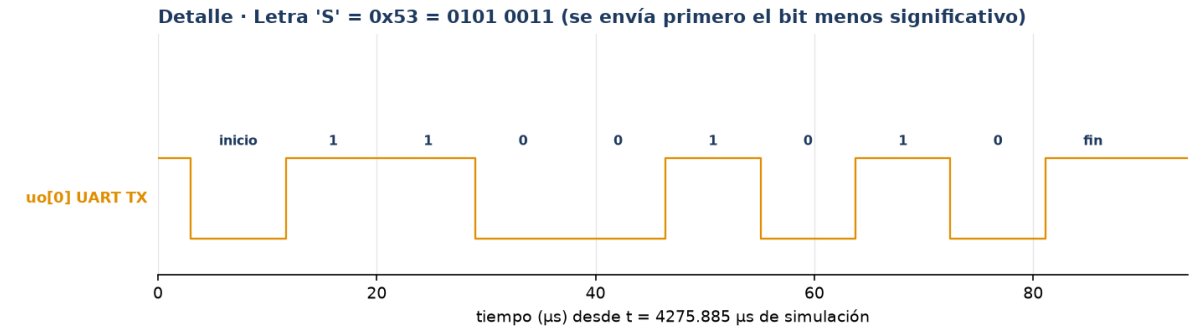
Medida	Esperado	Medido
Comando + dirección por MOSI	0x03000000	0x03000000
Período de SCK	80 ns (12,5 MHz)	80.0 ns
Palabra por MISO	1ª instrucción	0x17018010 (0x10800117, little-endian)

Resultado: **PASS**.

5.3 Arranque completo y mensaje por la UART



Vista general del arranque.

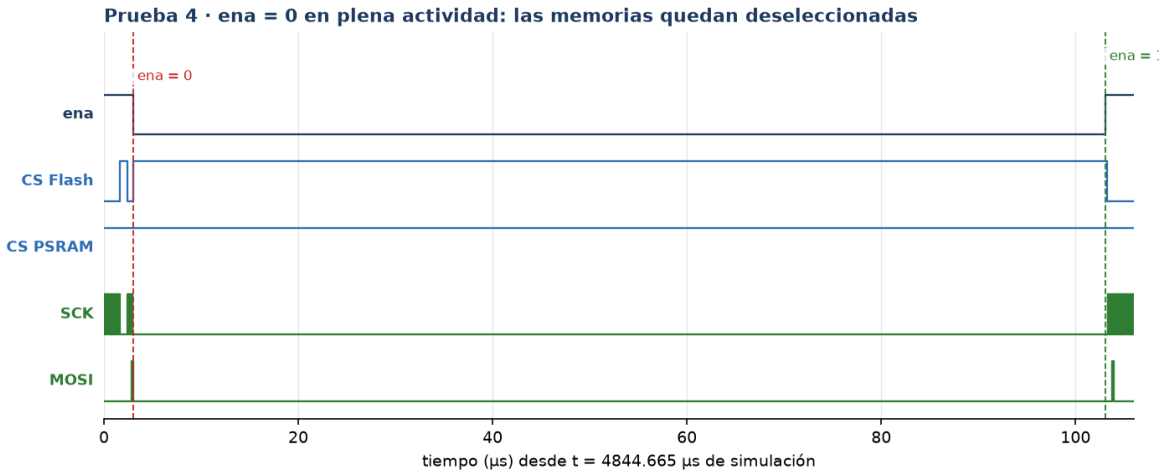


La letra 'S' decodificada en uo[0].

Medida	Esperado	Medido
Texto por uo[0]	"SML1\n"	"SML1\n"
Tiempo hasta la primera letra	-	4.27 ms
uo[4:1] al final	19 & 0xF = 3	3
uo[5] (done, pegajoso)	1	1

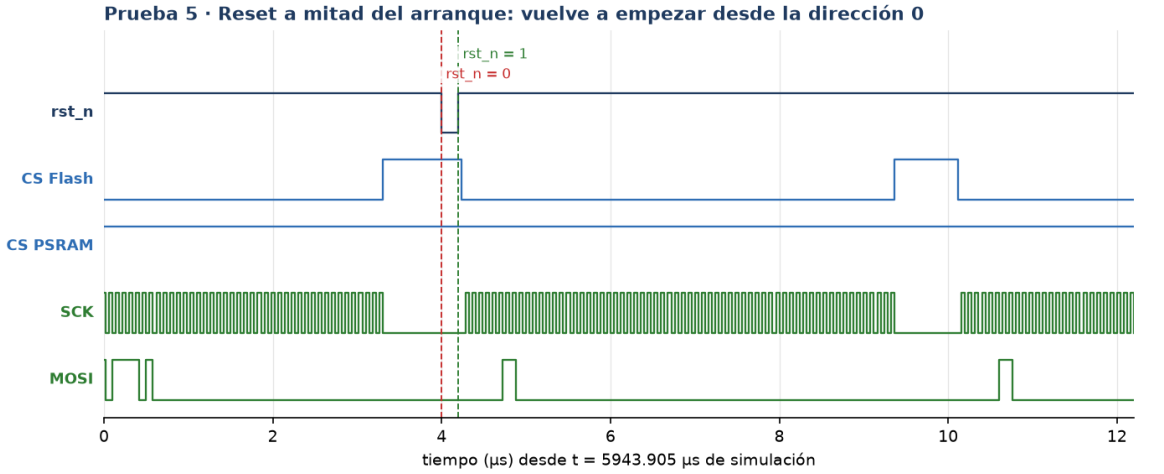
Resultado: **PASS**.

5.4 ena = 0 congela la memoria



Resultado: **PASS**. En 5 000 ciclos con ena = 0, ningún CS bajó y SCK no se movió.

5.5 Reset a mitad del arranque



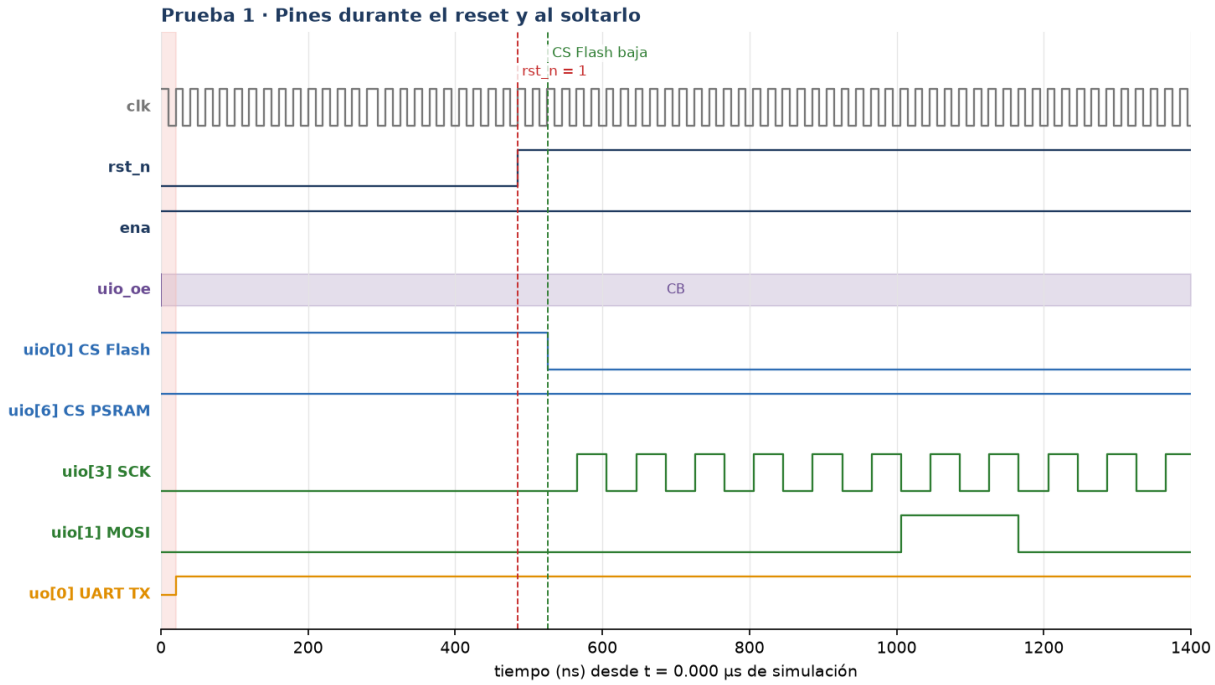
Resultado: **PASS**. Vuelve a empezar en la dirección 0 y termina enviando "SML1\n".

6. Gate-level

Sinteticé el diseño final con Yosys y la biblioteca real de SkyWater (8 108 compuertas, sin la celda de compuerta de reloj) y corrí las mismas 5 pruebas sobre ese netlist:

Medida	RTL	Gate-level
Texto por UART	SML1\n	SML1\n
Tiempo hasta la primera letra	4.2731 ms	4.2731 ms
Primer comando SPI	0x03000000	0x03000000
uo[4:1] final	3	3
Reset a mitad de arranque	SML1\n	SML1\n

Resultado: **5/5 PASS**. Además, GitHub Actions ejecuta la prueba gate-level oficial sobre el netlist final (sección 2.6).



Gate-level: uo[0] está indefinido (franja rosa) solo durante los primeros 20 ns, hasta el primer flanco de reloj.

7. Área, tiempos y lint

Dato	Antes	Después
Celdas (síntesis Yosys)	8 032	8 108
Área de compuertas (antes de colocar)	65 001 µm²	65 507 µm²
Flip-flops	1 609	1 626
Relojes	2 (clk y cpu_clk con compuerta)	1 (clk)
Camino más lento estimado (esquina típica)	9,04 ns	7,49 ns
Lint Verilator (errores / avisos)	0 / 5	0 / 5

Diseño físico final (LibreLane): 9 026 celdas estándar en un core de 110 874 µm², **82,9 % de utilización**. Incluye 76 celdas de retardo para hold. El tamaño 3x2 cierra, aunque por encima del 70 % recomendado: queda poco espacio para funciones nuevas.

8. Lo que no se pudo probar aquí

Qué	Por qué	Qué hacer
Descargar los archivos del diseño físico (GDS, netlist, reportes)	La red de esta sesión bloquea la descarga de los *artifacts* de GitHub.	Descargarlos desde la pestaña Actions (sección 9).
FPGA	No hay placa conectada.	Workflow fpga.yaml + placa ICE40.
Chip real, 3,3 V, Pmod físico	Necesita hardware.	Laboratorio.

9. Entregables

Entregable	Dónde está
Paquete de envío a Tiny Tapeout (**tt_submission**: GDS, LEF, netlist)	github.com/ric1806/tiny/actions/runs/36503628717 archivo **tt_submission**
Imagen del chip (**gds_render**)	github.com/ric1806/tiny/actions/runs/36503628717 archivo **gds_render**
Reportes del diseño físico (**GDS_logs**)	github.com/ric1806/tiny/actions/runs/36503628717 archivo **GDS_logs**
Resultado gate-level oficial y precheck	github.com/ric1806/tiny/actions/runs/36503628717 archivos **gatelevel_test_results** y **precheck_reports**
Hoja de datos del proyecto (datasheet)	github.com/ric1806/tiny/actions/runs/36503628672 archivo **PDF**
Código fuente, info.yaml, docs/info.md	Repositorio, rama claude/hola-wl31sp
Testbenches originales y actuales	Paquete Testbenches_SERV_TinyML.zip + carpeta test/
Guía del proyecto (actualizada)	docs/Guia_Proyecto_SERV_TinyML.pdf
Este informe	docs/Pruebas_SERV_TinyML.pdf
Resumen de cambios	docs/Cambios_SERV_TinyML.pdf

> **Ojo**
>
> Para enviar a Tiny Tapeout se indica el repositorio en app.tinytapeout.com, y la plataforma usa el resultado de GitHub Actions. Todo el trabajo está en la rama `claude/hola-wl31sp`; normalmente se pasa a la rama principal (main) antes de enviarlo. El envío y el pago los hacen ustedes.

10. Cómo repetir las pruebas

```
cd test
make -f Makefile           # top module
make -f Makefile.mac       # MAC
make -f Makefile.spi       # lector SPI
make -f Makefile.mem       # puente de memoria
make -f Makefile.serv      # SERV con memoria ideal
make -f Makefile.extmem    # arranque desde Flash/PSRAM
make -f Makefile.board     # chip completo (placa)
```

Cada una termina con una tabla; todas las filas deben decir **PASS**. El diseño físico se repite solo en GitHub Actions con cada `git push`.